

MightyMAK

Front-End Build Plan

A friendly web app for generating AI videos with Seedance 2.0 — this document scopes the **front-end only**: every screen, the generation studio, and the asset library, running on realistic **mock data** with a **simulated** generation flow. No real backend, billing, or model calls yet.

Scope	Front-end UI + mocked data + simulated generation
Stack	Next.js (App Router, TS) · Tailwind · shadcn/ui · Framer Motion
Model	Seedance 2.0 (mocked now; wired to BytePlus later)
Date	17 June 2026

WHAT WE'RE BUILDING FIRST

MightyMAK is “Seedance, but simple”: people sign up, buy a package, and generate AI videos — keeping every clip and reusable asset in a personal library. The full product needs auth, payments, storage, and the BytePlus video API. **This phase builds none of that.** It builds the entire interface, fed by mock data and a fake generation engine, so we can see, click, and refine the whole experience before spending a cent on infrastructure or model calls.

Why front-end first

You get a clickable, demo-ready product in days — validate the look, the flows, and the “feel” of generating a video — then we plug the real backend into clearly-marked seams without redesigning anything.

In scope vs. out of scope (this phase)

IN — we build this now	OUT — comes in the backend phase
▪ Every screen & the full navigation shell ▪ The Generation Studio (the hero screen) ▪ Simulated generation: progress → sample video ▪ Library: browse, preview, reuse, remix ▪ Pricing, billing & account screens (visual) ▪ Auth screens (sign in / up) as UI ▪ Design system, theming, responsive, animations ▪ Mock data persisted in the browser	▪ Supabase auth, database & storage ▪ Stripe checkout, subscriptions & webhooks ▪ Real BytePlus / Seedance 2.0 API calls ▪ Server-side credit ledger & holds ▪ Webhooks, cron pollers, re-hosting ▪ Real file uploads & signed URLs ▪ Admin tooling & ops

THE TOOLKIT

Lean, modern, and the same foundation the full product will use — so nothing is throwaway.

Next.js (App Router)	TypeScript	Tailwind CSS	shadcn/ui	Framer Motion	Zustand
Layer	Choice	Why			
Framework	Next.js + React (App Router, TypeScript)	App structure & routing; later hosts the real API too			
Styling	Tailwind CSS + shadcn/ui components	Fast, consistent, accessible UI primitives			
Motion	Framer Motion	Smooth transitions, the generation progress feel			
State	Zustand + browser localStorage	Mock credits, library & jobs that persist on refresh			
Mock layer	Typed services/ + MockVideoProvider	The seam where the real backend drops in later			

Layer	Choice	Why
Media	A few royalty-free sample MP4s	Stand-ins for “generated” results

THE SCREENS

Route	Screen	What's on it
/	Landing	Hero, sample videos, value prop, CTA to sign up
/pricing	Pricing	Subscription plans + top-up packs (visual, non-functional)
/sign-in · /sign-up	Auth	Forms & social buttons as UI; mock “log in” routes into the app
/dashboard	Dashboard	Welcome, credit balance, recent videos, active jobs, quick-create
/studio	Generation Studio	The hero screen — compose & run a generation
/library	Library	Grid of generated videos + saved reference assets; collections
/generations/[id]	Result	Player, details, Download, Remix, Use-as-reference
/billing	Billing	Current plan, credit balance, buy credits, invoice list (mock)
/account	Account	Profile, preferences, theme

THE HERO SCREEN

Where a non-technical person composes a powerful multimodal generation — simply. A single, calm screen: write what you want, optionally add reference media, tune a few controls, watch the live cost, and hit Generate.

Compose ■ Prompt box — large, friendly, with example prompts ■ Reference tray — drop up to ~12 images / clips / audio ■ Roles per reference: subject, environment, motion, audio ■ First / last frame control for image-to-video ■ Pull references straight from your Library or upload new	Controls & run ■ Duration, aspect ratio, quality tier (Fast / Standard / Pro) ■ Native audio toggle ■ Live credit-cost estimate that updates as you change settings ■ Generate — deducts mock credits, opens the job monitor ■ Job monitor — queued → rendering → done, with progress
--	---

Simulated generation (the magic, faked)
Pressing Generate runs a MockVideoProvider: it shows realistic queued → rendering → complete states over a few seconds, then returns a sample MP4 and drops it into the Library — exactly as the real flow will. The interface can't tell the difference, which is the whole point: swap the mock for BytePlus later and the UI is unchanged.

KEEP & REUSE

Everything the user makes or uploads lives here — and feeds the next generation.

- **Grid** of generated videos and saved reference assets, with hover-preview
- **Filters & collections** (folders) to stay organized
- **Result view**: full player, prompt & settings used, download
- **Remix** — reopens the Studio pre-filled with that video's prompt, refs & settings
- **Use as reference** — attach any past asset into a new generation
- Rename, delete, organize — all persisted in the browser so it feels real

MONEY (VISUAL ONLY)

The credit and plan screens are fully designed but **non-transactional** this phase: “Buy” just tops up a mock balance so the rest of the app can be exercised. Numbers below are placeholders to design against — we finalize them when real costs are confirmed.

Type	Name	Price	Credits
Subscription	Starter	\$12 / mo	1,000 credits / month
Subscription	Creator	\$39 / mo	4,000 credits / month
Subscription	Studio	\$99 / mo	12,000 credits / month
Top-up pack	Mini	\$7	500 credits (no expiry)
Top-up pack	Plus	\$20	1,600 credits (no expiry)
Top-up pack	Mega	\$60	5,500 credits (no expiry)

A credit-cost estimate (e.g. duration × quality tier) is shown live in the Studio so users always understand what a generation will cost before running it.

HOW IT'S WIRED

The single most important design choice: all data and actions go through a typed **service interface**. Today a mock implementation backs it; later the real Supabase / Stripe / BytePlus implementation drops in behind the same interface — **zero UI rewrites**.

Service	Now (mock)	Later (real)
auth	Fake session (local toggle / simple form)	Supabase Auth (email + Google)
credits	Mock balance in Zustand + localStorage	Server credit ledger with holds/settle
generations	MockVideoProvider timers + sample MP4	BytePlus Seedance 2.0 async jobs
library / assets	Seeded mock items in localStorage	Supabase Postgres + Storage
billing	"Buy" adds mock credits instantly	Stripe Checkout + webhooks

Design principle

The UI never talks to a provider directly — only to services/. That keeps this front-end phase honest about data shapes, so backend wiring is a connection job, not a redesign.

THE PLAN

1

- Create the Next.js + TypeScript + Tailwind app; install shadcn/ui
- Set fonts, colors, dark/light theme tokens, base components & layout
- Outcome: a styled empty app that builds and runs

2

- Sidebar, top bar, persistent credit-balance widget, routing between sections
- Landing page + pricing page (static, with sample videos)
- Outcome: you can navigate the whole product skeleton

3

- Build the composer, controls, live cost estimate, Generate button
- Wire the MockVideoProvider: progress states → sample result
- Outcome: run a fake generation end-to-end and see a video appear

4

- Library grid, collections, result/player view
- Remix (pre-fill Studio) and Use-as-reference flows; localStorage persistence
- Outcome: a generated clip can be browsed, remixed, and reused

5

- Plan/pack cards, mock “buy” that tops up credits, invoice list
- Sign in / up screens and account/profile settings as UI
- Outcome: the money & account surfaces are fully clickable

6

- Empty / loading / error states, out-of-credits prompts
- Animations, mobile responsive, accessibility pass
- Outcome: a demo-ready, refined product

DONE LOOKS LIKE

Verify (one sitting, desktop + mobile):

- Land on the homepage → “sign up” → reach the dashboard
- Open the Studio, write a prompt, add a reference, watch the cost update
- Generate → see progress → a sample video lands in the Library
- Remix it, and buy mock credits when the balance runs low
- Click every screen; check it looks right on a phone

Then: backend phase

Once you're happy with the look and flow, we connect the seams — Supabase (auth, data, storage), Stripe (real subscriptions + top-ups), and the BytePlus Seedance 2.0 API — behind the same service interfaces, turning the simulation into the real thing.

One assumption worth confirming: this phase treats the backend as fully mocked (no real logins, payments, or video calls). If you'd rather wire one real piece early (e.g. real sign-in), say the word and we'll fold it into step 1.